

Automating Email OTP Login with IMAP (Playwright)

A reusable pattern for testing apps that require a one-time code sent by email: **log in once per test run, read the OTP from a real Gmail inbox over IMAP, save the session, and let every test reuse it.**

Individual tests never deal with OTP at all.

The pattern at a glance

```
setup project (runs once, before all tests)
  1. Open login page, submit credentials
  2. Record a timestamp BEFORE submitting
  3. Poll the inbox over IMAP for an email newer than the timestamp
  4. Extract the 6-digit code, type it into the OTP inputs
  5. Verify success, save session → storageState.json

test projects
  - Load storageState.json → start already authenticated
```

One-time setup

1. **Dedicated Gmail account** with 2-Step Verification enabled.
2. **App Password**: Google Account → Security → 2-Step Verification → App passwords. Copy the 16-character password (no spaces). The normal account password will *not* work for IMAP.
3. **Enable IMAP**: Gmail Settings → Forwarding and POP/IMAP → Enable IMAP.
4. **Store credentials in** `.env` (never commit it):

```
GMAIL_USER=your.test.inbox@gmail.com
GMAIL_APP_PASSWORD=abcdefghijklmnop
```

5. Install the IMAP client: `npm i -D imapflow` and load env vars with `import 'dotenv/config'` in your Playwright config.

Step 1 — Run login once, before all tests

Use a Playwright **setup project** and make every browser project depend on it. (Prefer this over `globalSetup` — setup projects also run in UI mode and the VS Code extension.)

```
// playwright.config.ts
projects: [
  {
    name: 'setup',
    testMatch: /auth\.setup\.ts/,
    use: { storageState: { cookies: [], origins: [] } }, // start clean
  },
  {
    name: 'chromium',
    dependencies: ['setup'],
  },
],
use: {
  storageState: 'storageState.json', // all tests reuse the saved session
},
```

```
// tests/auth.setup.ts
setup('authenticate', async ({ page }) => {
  await loginWithEmail0tp(page, credentials);
  await page.context().storageState({ path: 'storageState.json' });
});
```

Step 2 — Capture a timestamp, then submit

```
const loginTime = new Date(); // BEFORE clicking Login
await loginButton.click();
```

Only emails that arrived **after** this moment are considered. This one line is what prevents picking up a stale OTP from a previous run.

Step 3 — Poll the inbox over IMAP

```
import { ImapFlow } from 'imapflow';

export async function fetchOtpFromEmail(afterTimestamp: Date): Promise<string> {
  const deadline = Date.now() + 60_000;          // give the email 60s to arrive

  while (Date.now() < deadline) {
    const client = new ImapFlow({
      host: 'imap.gmail.com',
      port: 993,
      secure: true,
      auth: {
        user: process.env.GMAIL_USER!,
        pass: process.env.GMAIL_APP_PASSWORD!,
      },
      logger: false,
    });
    await client.connect();

    try {
      await client.mailboxOpen('INBOX');
      const uids = await client.search({ since: afterTimestamp });

      for (const uid of [...uids].reverse()) {      // newest first
        const msg = await client.fetchOne(String(uid), { envelope: true, source: true });

        // IMAP 'since' is date-granular – re-check the actual time
        if (!msg?.envelope?.date || new Date(msg.envelope.date) < afterTimestamp) continue;

        const text = extractPlainText(msg.source.toString('utf8'));

        // Prefer a 6-digit number near a keyword; fall back to any 6-digit number
        const match =
          text.match(/(?:otp|code|verification)[^a-z\d]{0,60}(\d{6})/i) ??
          text.match(/\b(\d{6})\b/);
        if (match) return match[1];
      }
    } finally {
      await client.logout();
    }

    await new Promise((r) => setTimeout(r, 3_000)); // poll every 3s
  }
}
```

```
throw new Error('OTP email not received within 60s');
}
```

`extractPlainText` should decode the `text/plain` MIME part, handling base64 and quoted-printable transfer encodings — matching the raw source directly breaks when encoding splits the code across lines.

Step 4 — Type the code digit-by-digit

```
for (let i = 0; i < 6; i++) {
  const input = page.locator(`#otp${i + 1}`);      // adjust to your app's inputs
  await input.click();
  await input.fill('');                             // clear leftovers from a rejected try
  await input.pressSequentially(code[i], { delay: 50 });
}
```

Use `pressSequentially()`, not `fill()` — per-digit OTP inputs usually listen for keyboard events, and `fill()` can silently no-op on them.

Step 5 — Detect success reliably, retry on rejection

- **Detect success by a logged-in-only UI element** (e.g. a sidebar link), not by URL. Many SPAs render the dashboard after OTP with *no URL change*, so `waitForURL()` times out even on success. Avoid `networkidle` too — dashboards that poll never go idle.
- **Retry up to 3 times.** If the app doesn't auto-resend, click *Resend* yourself, and reset the fetch cutoff to `new Date(Date.now() - 10_000)` — the 10s slack absorbs clock skew between your machine and the mail server, while the rejected email (older still) can't be re-picked.
- **Give the setup test a generous timeout** (e.g. `setup.setTimeout(240_000)`) so retries aren't killed by the default test timeout.

Gotchas

- **IMAP rejects your login** → you used the account password instead of an App Password, or IMAP isn't enabled.
- **Email exists but is never found** → check the mailbox. Gmail hides Promotions-tab emails from `INBOX` over IMAP; open `[Gmail]/All Mail` to see everything.
- **Wrong code extracted** → dump the raw email source to a file during development and tune the regex against it.
- **Shared inboxes pile up** → add a best-effort cleanup before login that moves old test emails to Trash, scoped strictly to your app's sender address.
- **Parallel workers** → keep OTP login in the single setup project only; if multiple workers log in independently they race each other for codes.